

Streams Node.js

Traiter de gros volumes sans saturer la mémoire

Module 2 / 7

Objectifs du module

Durée : 3h30

- Distinguer et utiliser les **Readable** et **Writable** streams
- Mettre en oeuvre les **Duplex** et **Transform** streams
- Chaîner des streams proprement avec **stream.pipeline** et une gestion d'erreurs robuste

À l'issue de ce module, vous saurez traiter des fichiers de plusieurs gigaoctets avec une empreinte mémoire constante.

Plan du module

1. Writable et Readable streams

modes flowing/paused, highWaterMark, events `data` / `end` / `drain`

2. Duplex streams

lecture et écriture sur un même objet

3. Transform streams

transformation à la volée, chaînage avec `stream.pipeline` et gestion d'erreurs

1. Writable et Readable streams

Le concept : pourquoi des streams ?

Charger un fichier de 2 Go en mémoire avec `fs.readFile` :

- l'intégralité du contenu réside en RAM avant tout traitement
- pic mémoire proportionnel à la taille du fichier
- risque de crash (`JavaScript heap out of memory`)

Un **stream** traite la donnée **par morceaux (chunks)** : l'empreinte mémoire reste **bornée** quelle que soit la taille de la source.

Les 4 types : **Readable**, **Writable**, **Duplex**, **Transform**.

Readable stream

Source de données que l'on consomme par morceaux.

Exemples : `fs.createReadStream`, une réponse HTTP entrante, `process.stdin`.

Deux modes de lecture :

- **paused** : on tire la donnée avec `.read()`
- **flowing** : la donnée est poussée via l'événement `data`

```
const fs = require('node:fs');

const rs = fs.createReadStream(
  './big.log',
  { encoding: 'utf8', highWaterMark: 64 * 1024 }
);

rs.on('data', (chunk) => {
  console.log('chunk', chunk.length);
});
rs.on('end', () => console.log('terminé'));
rs.on('error', (err) => console.error(err));
```

Modes flowing et paused

paused (par défaut à la création)

```
rs.on('readable', () => {  
  let chunk;  
  while ((chunk = rs.read()) !== null) {  
    process(chunk);  
  }  
});
```

On contrôle explicitement le rythme.

Bascule automatique en flowing dès qu'un listener `data` est attaché ou qu'un `pipe()` est branché.

flowing

```
rs.on('data', (chunk) => process(chunk));
```

Attacher un listener `data` (ou appeler `.resume()`) bascule en flowing.

`.pause()` revient en paused.

Writable stream

Destination vers laquelle on écrit par morceaux.

Exemples : `fs.createWriteStream`, une réponse HTTP sortante, `process.stdout`.

`write()` retourne un booléen :

- `true` : on peut continuer à écrire
- `false` : le buffer interne est plein, il faut attendre

```
const ws = fs.createWriteStream('./out.txt');

const ok = ws.write('ligne 1\n');
if (!ok) {
  // buffer plein : attendre 'drain'
  ws.once('drain', () => {
    ws.write('ligne 2\n');
  });
}

ws.end('dernière ligne\n');
ws.on('finish', () => console.log('écrit'));
```


highWaterMark et backpressure

- **highWaterMark** : seuil (en octets, 64 Ko par défaut pour les fichiers) du buffer interne avant que `write()` retourne `false`.
- Lorsque `write()` retourne `false`, le producteur **doit s'arrêter** et attendre l'événement `drain`.
- Ce mécanisme est la **backpressure** : il évite qu'un producteur rapide ne sature un consommateur lent.

Gérer la backpressure à la main est fastidieux et source de bugs.

`pipe()` et `pipeline()` la gèrent **automatiquement**.

pipe() : brancher Readable vers Writable

```
const fs = require('node:fs');
const zlib = require('node:zlib');

fs.createReadStream('./input.txt')
  .pipe(zlib.createGzip())
  .pipe(fs.createWriteStream('./input.txt.gz'));
```

`pipe()` gère la backpressure automatiquement, mais **propage mal les erreurs** et ne ferme pas tous les streams en cas d'échec.

En production, préférer `stream.pipeline` (voir section 3).

2. Duplex streams

Le concept : Duplex

Un **Duplex** stream est à la fois **Readable** et **Writable**, mais les deux côtés sont **indépendants** (deux buffers internes distincts).

Exemple canonique : une **socket TCP** (`net.Socket`) — on lit ce qui arrive du réseau, on écrit ce qu'on envoie.

À ne pas confondre avec un Transform : dans un Duplex, ce qu'on écrit n'a **aucun rapport mécanique** avec ce qu'on lit.

Implémenter un Duplex

On fournit deux méthodes :

- `_read(size)` : alimente le côté lecture
- `_write(chunk, enc, cb)` : consomme le côté écriture

Les deux flux vivent leur vie séparément.

```
const { Duplex } = require('node:stream');

class EchoStore extends Duplex {
  constructor(opts) {
    super(opts);
    this.data = [];
  }
  _write(chunk, enc, cb) {
    this.data.push(chunk);
    cb();
  }
  _read() {
    const chunk = this.data.shift();
    this.push(chunk ?? null);
  }
}
```

Cas d'usage typiques

- **net.Socket** : communication TCP bidirectionnelle
- **zlib** en mode interactif
- **crypto** (`Cipher` / `Decipher`) : ce sont en réalité des Transform, cas particulier de Duplex
- adaptation entre deux protocoles (proxy, tunnel)

En pratique, on consomme souvent des Duplex existants ; on en écrit rarement à la main, sauf pour des protocoles réseau personnalisés.

3. Transform streams et pipeline

Le concept : Transform

Un **Transform** est un Duplex où la sortie est **fonction de l'entrée** : ce qu'on lit est le résultat de la transformation de ce qu'on écrit.

Exemples natifs : `zlib.createGzip()`, `crypto.createCipheriv()`.

Une seule méthode à implémenter : `_transform(chunk, encoding, callback)`.

C'est l'outil idéal pour transformer un flux **à la volée** : compression, chiffrement, parsing CSV, conversion d'encodage.

Implémenter un Transform

```
const { Transform } = require('node:stream');

class Upper extends Transform {
  _transform(chunk, enc, cb) {
    this.push(
      chunk.toString().toUpperCase()
    );
    cb();
  }
}
```

- `chunk` : morceau entrant (Buffer ou string)
- `this.push(data)` : émet vers la sortie
- `cb()` : signale la fin du traitement du chunk
- `cb(err)` : propage une erreur
- `_flush(cb)` : optionnel, pour vider un reste en fin de flux

`objectMode: true` permet de faire circuler des objets plutôt que des Buffers.

stream.pipeline : le chaînage propre

```
const { pipeline } = require('node:stream/promises');
const fs = require('node:fs');
const zlib = require('node:zlib');

await pipeline(
  fs.createReadStream('./input.txt'),
  zlib.createGzip(),
  fs.createWriteStream('./input.txt.gz')
);
console.log('Pipeline terminée avec succès');
```

`pipeline` gère la **backpressure**, **propage les erreurs**, et **ferme proprement tous les streams** en cas d'échec. La version `stream/promises` retourne une Promise.

Gestion d'erreurs : pipeline vs pipe

pipe() — fragile

```
rs.pipe(transform).pipe(ws);  
// chaque stream doit gérer  
// son propre 'error', sinon  
// fuite de descripteurs  
rs.on('error', onErr);  
transform.on('error', onErr);  
ws.on('error', onErr);
```

pipeline() — robuste

```
try {  
  await pipeline(rs, transform, ws);  
} catch (err) {  
  // une seule erreur centralisée  
  // tous les streams sont fermés  
  console.error('Échec : ', err);  
}
```

Règle : en production, toujours `pipeline` plutôt que `pipe` enchaînés.

Démonstration

Streamer un gros fichier via Express en réponse HTTP, sans le charger en mémoire, et observer une **consommation mémoire stable**.

Voir `demos/demo-02.md`

Exercice

Écrire un **Transform stream** qui transforme un flux CSV à la volée : mise en majuscules d'une colonne et filtrage de lignes selon une condition.

Difficulté : 3/5

Fichier : `exercices/exercice-02.md`

Travaux pratiques

Exposer un endpoint Express de traitement de gros fichier en streaming : upload → transform → écriture/réponse, via `stream.pipeline` avec gestion d'erreurs complète.

Difficulté : 4/5 — Durée : environ 1h30

Fichier : `tp/tp-02.md`

Récapitulatif

- Un **Readable** est une source, un **Writable** une destination ; on les consomme par chunks.
- **flowing** vs **paused**, **highWaterMark** et **backpressure** (`drain`) bornent la mémoire.
- **Duplex** : lecture et écriture indépendantes ; **Transform** : sortie fonction de l'entrée.
- `stream.pipeline` : chaînage propre, backpressure et erreurs gérées.

Cheatsheet : `cheatsheet/module-02.md`

Questions ?

Module suivant : **Tests avec Node.js**